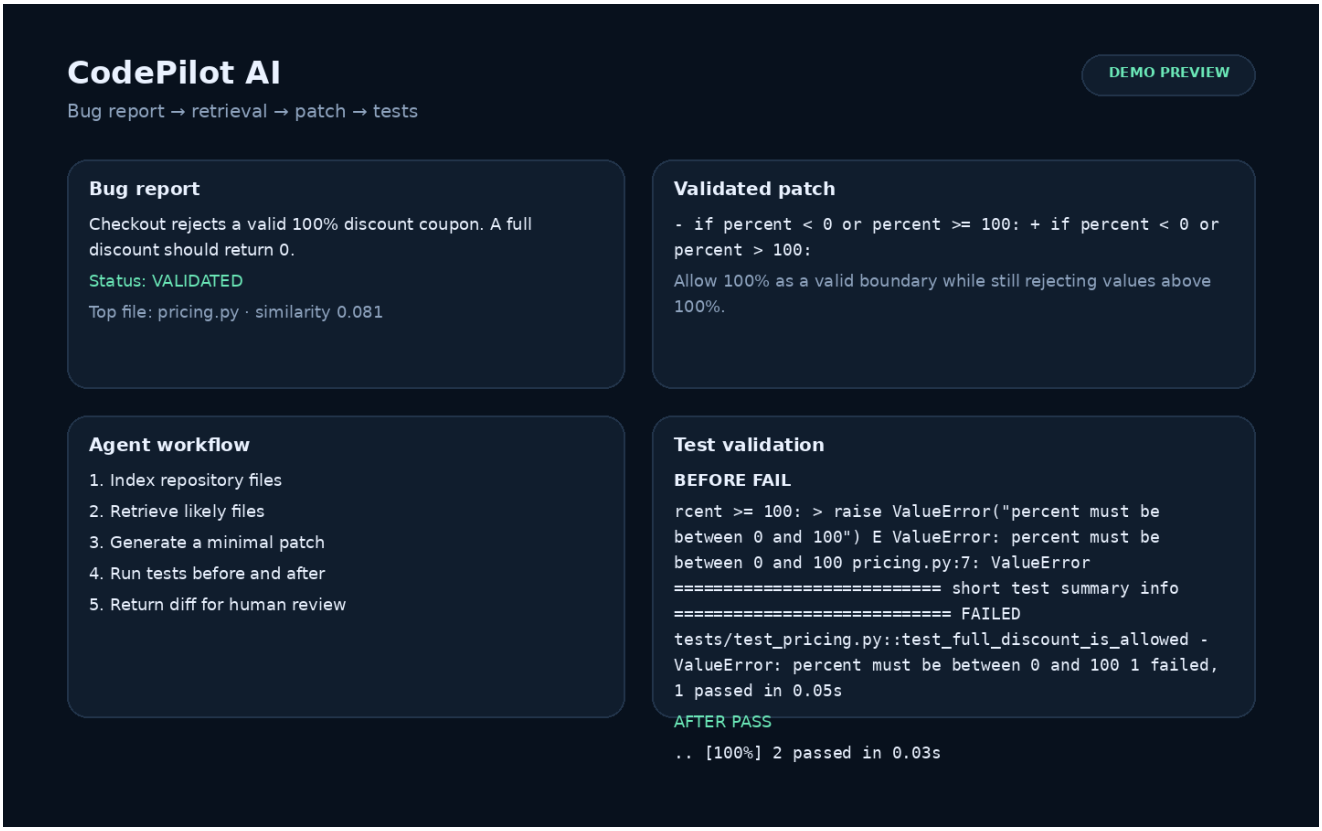
# CodePilot AI

Autonomous Software Engineering Agent

Python • FastAPI • Vectorized Code Retrieval • Pytest • Docker • CI/CD




Representative demo preview. Run the included application for the interactive version.

# 1. Problem and Solution

Developers spend time translating bug reports into code locations, preparing changes, and validating regressions. CodePilot demonstrates a controlled agent loop that automates the repetitive steps while keeping a human-review boundary.

## Why this is strong on an AI/ML resume

The project is built as a small product rather than only a notebook. It includes a backend API, interactive frontend, repeatable local data, automated tests, Docker packaging, CI configuration, demo instructions, and clear model/engineering boundaries.

## Core components

| Component | Responsibility |
| --- | --- |
| Repository Retriever | Indexes source files and ranks likely code using TF-IDF vectors and cosine similarity. |
| Patch Planner | Creates a minimal safe change in the local demo and exposes a provider-neutral extension point for stronger planning. |
| Sandbox Validator | Copies the repository to a temporary workspace and runs tests before and after the candidate patch. |
| Review Layer | Returns retrieved files, reasoning, a unified diff and validation status instead of silently modifying source. |

# 2. Architecture and Data Flow

- **Indexer/Retriever:** sparse vector representation with TF-IDF and cosine similarity. - **Patch planner:** deterministic safe rule in the local demo; provider-neutral LLM adapter is isolated behind providers/. - **Sandbox:** copies the repository to a temporary directory before changing files. - **Validator:** runs pytest with a timeout and reports before/after state. - **Guardrail:** never modifies the bundled source during analysis; only returns a patch for review.

## Engineering decisions

- Keep the core demo fully local so it is reliable during interviews.
- Separate model logic from API/UI so algorithms can be upgraded without rewriting the product surface.
- Use explicit validation and automated tests so the demo can be explained as an engineering system, not only an ML experiment.
- Package with Docker and CI configuration to demonstrate repeatability.

## What “production-style” means here

The repositories demonstrate production patterns, but they are portfolio systems, not claims of enterprise production deployment. A real deployment would add identity/access management, secret management, persistent observability, load testing, threat modeling, data governance, backup/rollback processes and infrastructure-specific scaling.

# 3. Live Demo Walkthrough

**1.** Open the dashboard and keep the pre-filled 100% coupon bug report.

**2.** Run analysis. Show that pricing.py is ranked first.

**3.** Show the one-line boundary-condition diff.

**4.** Compare failing tests before the patch with passing tests after it.

**5.** Explain that production deployment would add Git provider integration, containerized job isolation and policy checks.

## What to say while demoing

> *CodePilot AI automates the repetitive part of bug resolution. A developer enters a bug report. The service vectorizes repository files, retrieves the most relevant code, generates a minimal patch, applies it only inside a temporary sandbox, runs automated tests, and returns a unified diff for human review. In my demo, a 100% coupon is incorrectly rejected. The original tests fail, the generated boundary-condition fix is applied, and the tests pass. I focused on validation and human approval rather than blindly editing source code. 1. Problem: developers spend time locating code and validating small bugs. 2. Architecture: retrieval, planning, sandboxing, tests, review. 3. Live demo: run the bundled coupon bug. 4. Engineering choices: FastAPI, vector search, pytest subprocess isolation, Docker/CI. 5. Limitations: local deterministic patch rule; future model integration, GitHub PRs, policy checks and observability.*

## Demo failure fallback

Keep `demo_output.json` and the preview image in the repository. If a live environment fails, walk through the saved output, explain the expected API call, then show the unit tests. This demonstrates preparedness without pretending the failed live run succeeded.

# 4. Resume Bullets

- Built an autonomous software-engineering agent that indexes a repository, retrieves bug-relevant files using vectorized code search, generates minimal patches, and validates changes with automated tests.
- Implemented a FastAPI workflow for bug triage, patch reasoning, unified-diff generation, sandboxed test execution, and human approval before code changes are accepted.
- Designed provider-agnostic LLM/RAG extension points, Dockerized deployment, API health checks, and CI testing for a production-style engineering workflow.

## Important claim rule

Only claim technologies you can run and explain. Optional modules such as YOLO, PyTorch two-tower models, FAISS, MLflow or Airflow are included as upgrade paths where noted; do not say you trained/deployed them until you actually execute that path and record evidence/metrics.

## Metrics to discuss

| | | | | |
|---|---|---|---|---|
| Retrieval recall@K | Patch test-pass rate | Regression rate | Human acceptance rate | Median time-to-validated-patch |

# 5. Interview Questions

## Why sandbox execution?

Generated changes should not modify the source repository before validation and approval.

## How would you scale it?

Persist embeddings in a vector index, incremental re-indexing on commits, isolate test jobs in containers, queue long-running tasks, cache repository metadata and stream status events.

## What metrics matter?

File-retrieval recall@k, patch pass rate, regression rate, time-to-fix, false-positive patch rate and human acceptance rate.

## Questions you should be ready for

- Why did you choose this algorithm instead of an alternative?
- What is the biggest failure mode in the current implementation?
- How would the architecture change at 100x traffic/data size?
- Which offline and online metrics would you monitor?
- What did you personally implement, test and debug?

# 6. Upgrade Roadmap

- Add authentication and role-based access.
- Persist model/retrieval artifacts with explicit versioning.
- Add structured observability: request logs, latency, error rates and model metrics.
- Create a small evaluation suite with expected outputs and regression thresholds.
- Deploy to a cloud service and record the public demo URL in the README.
- Add one real dataset experiment and document the training/evaluation results.

## Final presentation advice

Do not memorize only the feature list. Be able to draw the data flow, explain one algorithm choice, show one failure case, quote one meaningful metric and describe one scalability improvement. That is what turns the project from a resume line into an interview asset.